// QE LEARNING PATH · PHASE P04 · ~28 H

API testing with requests

Eight tasks that take you from `curl -i` to contract tests: HTTP semantics, CRUD and negative testing, a typed client layer, **Docker and Compose as a user**, bearer auth, pagination and breaking-change detection. The test examples **run real pytest and requests in your browser**, against a simulated Restful-Booker.

- 🧠 **Level 1 — Crack It** · the 🟢 low example
- 🚀 **Level 2 — Impress Them** · the 🔵 medium example
- 🏆 **Level 3 — Hero Mode** · the 🟣 high/complex example

🟢 Zero · 🔵 Hero · 🟣 Super Hero task levels

Z · H · SH interview-question levels

+ ADDED not in P04-api-requests.md — added to fill a gap

P04 — THE PHASE AT A GLANCE

🗺️ Eight tasks, ~28 hours

Goal: test REST APIs like a professional: HTTP fluency, CRUD and negative tests, a clean client layer, **running the apps locally with Docker**, schema validation and contract testing.

Apps: Restful-Booker, Toolshop API, Swagger Petstore, The Internet (Docker image).
Prerequisites: P03. **Time:** ~28 h.

ID	TASK	LEVEL	TIME	YOU DELIVER
P04-01	**HTTP fundamentals and manual API exploration**	🟢 Zero	2h	`docs/api-exploration.md`
P04-02	**First API tests with requests**	🟢 Zero	3h	`tests/api/booker/test_booking_basic.`
P04-03	**Auth, CRUD and negative testing**	🔵 Hero	4h	`tests/api/booker/test_booking_crud.p` `tests/api/booker/test_booking_negati`
P04-04	**API client layer and schema validation**	🔵 Hero	4h	`framework/api/base_client.py`, `framework/api/booker_client.py`, `framework/api/models/booking.py`
P04-05	**Docker as a user: images, containers, ports and volumes**	🟢 Zero	3h	`docs/docker-cheatsheet.md`
P04-06	**Run the demo stack locally with Compose**	🟢 Zero	3h	`docs/local-stack.md`, `Makefile` targe `stack-up`, `stack-down`, `stack-reset`
P04-07	**Toolshop API: bearer auth, pagination, filters, data factories**	🔵 Hero	4h	`framework/api/toolshop_client.py`, `framework/data/factories.py`, `tests/api/toolshop/`

ID	TASK	LEVEL	TIME	YOU DELIVER
P04-08	**Contract testing and API resilience**	Super Hero	5h	`tests/contract/`, `scripts/spec_diff.` `docs/contract-testing.md`

HOW THE RUN BUTTONS WORK HERE

The test examples run **real pytest 8.1 and requests 2.31** in your browser. There's no network inside the page, so calls to Restful-Booker are answered by a **simulation** that plugs into requests' transport layer: your test code runs unchanged. Pick options from the dropdown next to ▶ **Run pytest**. The curl, Bruno and Docker steps need your own terminal.

KEY

The fixtures these tests use — from P03-04, unchanged

P04-02 says to use the `http`, `settings` and `booker_url` fixtures from P03. Here they are, exactly as the P03 guide wrote them; the Run buttons put them in `tests/conftest.py` and `tests/api/conftest.py`.

tests/conftest.py

```python
# tests/conftest.py
import pytest, requests
from framework.config import get_settings

@pytest.fixture(scope="session")
def settings():
    return get_settings()

@pytest.fixture(scope="session")
def http():
    session = requests.Session()
    session.headers.update({"Accept": "application/json"})
    yield session                 # ← tests run here
    session.close()               # teardown, always
```

tests/api/conftest.py

```
# tests/api/conftest.py
import pytest

@pytest.fixture(scope="session")
def booker_url(settings):
    return settings.booker_url

@pytest.fixture(scope="session")
def booker_token(http, booker_url, settings):
    r = http.post(f"{booker_url}/auth", json={"username": "admin", "password":
"password123"},
                  timeout=settings.http_timeout)
    r.raise_for_status()
    return r.json()["token"]

@pytest.fixture
def booking(http, booker_url, booker_token, settings):
    from python_basics.checkout import build_booking   # P02-02 builder
    r = http.post(f"{booker_url}/booking", json=build_booking(),
timeout=settings.http_timeout)
    r.raise_for_status()
    created = r.json()                                  # {"bookingid": 123,
"booking": {...}}
    yield created
    http.delete(f"{booker_url}/booking/{created['bookingid']}",
                cookies={"token": booker_token}, timeout=settings.http_timeout)

# tests/api/conftest.py (continued)
import logging
import pytest

log = logging.getLogger("fixtures")

@pytest.fixture
def make_booking(http, booker_url, booker_token, settings, request):
    created: list[int] = []

    def _make(**overrides):
        from python_basics.checkout import build_booking
        r = http.post(f"{booker_url}/booking", json=build_booking(**overrides),
timeout=settings.http_timeout)
        r.raise_for_status()
        booking_id = r.json()["bookingid"]
        created.append(booking_id)
        log.info("[%s] created booking %s", request.node.name, booking_id)
        return r.json()

    yield _make

    for booking_id in reversed(created):                # LIFO cleanup
        resp = http.delete(f"{booker_url}/booking/{booking_id}",
                           cookies={"token": booker_token},
timeout=settings.http_timeout)
        if resp.status_code ⩾ 400:
            log.warning("cleanup failed for %s: %s", booking_id, resp.status_code)
```

framework/pytest_plugin.py

```python
# framework/pytest_plugin.py   (from P03-06: keep each phase's report on the test item)
import pytest

@pytest.hookimpl(wrapper=True)
def pytest_runtest_makereport(item, call):
    report = yield                      # let pytest build the report
    setattr(item, f"rep_{report.when}", report)   # rep_setup / rep_call / rep_teardown
    return report
```

KEY **The simulated Restful-Booker** + ADDED

It copies only the behaviours the guides rely on: `/ping` → 201, a token from `POST /auth`, 403 without the token for PUT/PATCH/DELETE, 201 for DELETE, 404 for unknown ids, 400 for invalid bodies (500 with `--buggy-booker`). Any other host is unreachable. A stand-in `framework/config.py` replaces pydantic-settings, and P02's reference code provides `build_booking()`.

```
"""A simulated Restful-Booker for the lesson's Run buttons (browser sandbox and local
checks).

It plugs into the real `requests` stack at the transport layer, so code that uses
requests.Session, cookies, Basic auth, hooks and retry adapters runs unchanged.
Requests to the Restful-Booker hosts below are answered from memory; any other host
gets a ConnectionError, because the sandbox has no network.

It copies the behaviours the guides rely on: /ping answers 201, POST /auth returns a
token, PUT/PATCH/DELETE need the token cookie or Basic admin:password123 (otherwise
403), DELETE answers 201, and unknown ids answer 404. Invalid bodies get 400, or 500
when installed with buggy=True, to show what a "never 5xx" guard is for.
"""
from __future__ import annotations

import base64
import copy
import json
import secrets
from urllib.parse import parse_qs, urlsplit

import requests
from requests.adapters import HTTPAdapter
from requests.structures import CaseInsensitiveDict

HOSTS = {"restful-booker.herokuapp.com", "localhost:3001", "127.0.0.1:3001",
"restful-booker:3001"}
REASONS = {200: "OK", 201: "Created", 400: "Bad Request", 403: "Forbidden", 404: "Not
Found",
           405: "Method Not Allowed", 500: "Internal Server Error"}
FIELDS = {"firstname": str, "lastname": str, "totalprice": int, "depositpaid": bool,
"bookingdates": dict}
BASIC = "Basic " + base64.b64encode(b"admin:password123").decode()
MALFORMED = object()


class FakeBooker:
    def __init__(self) -> None:
        self.buggy = False
        self.reset()

    def reset(self) -> None:
        self.tokens: set[str] = set()
        self.bookings: dict[int, dict] = {}
        self.next_id = 1
        for first, last in (("Sally", "Brown"), ("Jim", "Wilson")):          # a
little seed data
            self._store({"firstname": first, "lastname": last, "totalprice": 111,
```

```
"depositpaid": True,
                          "bookingdates": {"checkin": "2026-01-01", "checkout": "2026-
01-02"},
                          "additionalneeds": "Breakfast"})

    def _store(self, booking: dict) -> int:
        booking_id, self.next_id = self.next_id, self.next_id + 1
        self.bookings[booking_id] = copy.deepcopy(booking)
        return booking_id

    @staticmethod
    def _json(raw: object) -> object:
        if raw is None:
            return None
        try:
            return json.loads(raw.decode() if isinstance(raw, bytes) else raw)
        except ValueError:
            return MALFORMED

    @staticmethod
    def _invalid(body: object) -> bool:
        if not isinstance(body, dict):
            return True
        for name, kind in FIELDS.items():
            value = body.get(name)
            if value is None or not isinstance(value, kind) or (kind is int and
isinstance(value, bool)):
                return True
        return not all(isinstance(body["bookingdates"].get(k), str) for k in
("checkin", "checkout"))

    def _bad(self) -> tuple[int, str]:
        return (500, "Internal Server Error") if self.buggy else (400, "Bad Request")

    def _authorised(self, headers: CaseInsensitiveDict) -> bool:
        if headers.get("Authorization") == BASIC:
            return True
        pairs = (part.strip().split("=", 1) for part in headers.get("Cookie",
"").split(";") if "=" in part)
        return dict(pairs).get("token") in self.tokens

    def handle(self, method: str, url: str, headers: CaseInsensitiveDict, raw:
object) -> tuple[int, object]:
        parts = urlsplit(url)
        path = parts.path.rstrip("/") or "/"
        query = {key: values[0] for key, values in parse_qs(parts.query).items()}
        segments = path.strip("/").split("/")
        if path == "/ping":
            return 201, "Created"
        if path == "/auth" and method == "POST":
            if self._json(raw) == {"username": "admin", "password": "password123"}:
                token = secrets.token_hex(8)
                self.tokens.add(token)
                return 200, {"token": token}
            return 200, {"reason": "Bad credentials"}
        if segments[0] != "booking" or len(segments) > 2:
            return 404, "Not Found"
        if len(segments) == 1:
            if method == "GET":
                wanted = {k: v for k, v in query.items() if k in ("firstname",
"lastname")}
```

```
                return 200, [{"bookingid": i} for i, b in self.bookings.items()
                             if all(b.get(k) == v for k, v in wanted.items())]
            if method != "POST":
                return 405, "Method Not Allowed"
            body = self._json(raw)
            if body is MALFORMED:
                return 400, "Bad Request"
            if self._invalid(body):
                return self._bad()
            return 200, {"bookingid": self._store(body), "booking": body}
        if not segments[1].isdigit():
            return 404, "Not Found"
        booking_id = int(segments[1])
        if method == "GET":
            return (200, self.bookings[booking_id]) if booking_id in self.bookings
else (404, "Not Found")
        if method not in ("PUT", "PATCH", "DELETE"):
            return 405, "Method Not Allowed"
        if not self._authorised(headers):
            return 403, "Forbidden"
        if booking_id not in self.bookings:
            return 404, "Not Found"
        if method == "DELETE":
            del self.bookings[booking_id]
            return 201, "Created"
        body = self._json(raw)
        if body is MALFORMED or not isinstance(body, dict):
            return 400, "Bad Request"
        if method == "PATCH":
            merged = copy.deepcopy(self.bookings[booking_id])
            for key, value in body.items():
                if isinstance(value, dict) and isinstance(merged.get(key), dict):
                    merged[key].update(value)
                else:
                    merged[key] = value
            body = merged
        if self._invalid(body):
            return self._bad()
        self.bookings[booking_id] = copy.deepcopy(body)
        return 200, body


SERVER = FakeBooker()


def _send(adapter: HTTPAdapter, request: requests.PreparedRequest, **kwargs: object)
-> requests.Response:
    host = urlsplit(request.url).netloc
    if host not in HOSTS:
        raise requests.ConnectionError(
            f"lesson sandbox: no network, so {host} is unreachable (only the
simulated Restful-Booker answers)",
            request=request)
    status, payload = SERVER.handle(request.method, request.url, request.headers,
request.body)
    is_json = not isinstance(payload, str)
    response = requests.Response()
    response.status_code, response.reason = status, REASONS.get(status, "")
    response._content = (json.dumps(payload) if is_json else payload).encode()
    response._content_consumed = True
    response.headers = CaseInsensitiveDict({
```

```
        "Content-Type": f"{'application/json' if is_json else 'text/plain'};
charset=utf-8",
        "X-Simulated-By": "lesson sandbox"})
    response.encoding, response.url, response.request, response.connection = "utf-8",
request.url, request, adapter
    return response


def install(buggy: bool = False) -> FakeBooker:
    """Route Restful-Booker hosts to the simulation for every requests session in
this process."""
    SERVER.reset()
    SERVER.buggy = buggy
    HTTPAdapter.send = _send
    return SERVER
```

PRIMERS — FUNDAMENTALS

📚 One request, one response

Every API test in this phase sends a request and checks what comes back. Three ideas to have straight first: the **shape of an HTTP exchange**, the **requests API**, and the **layers** an API framework grows into.

⇄ Anatomy of an HTTP exchange

A request is a **method**, a **URL**, **headers** and an optional **body**. A response is a **status**, **headers** and a **body**. Read a response top to bottom: status first, then the headers that matter, then the body.

```
PUT /booking/42 HTTP/1.1                          ← method + path (+ ?query)
Host: restful-booker.herokuapp.com
Content-Type: application/json                    ← what I'm sending
Accept: application/json                          ← what I want back
Cookie: token=4f1c2a9b                            ← who I am

{"firstname": "Grace", "lastname": "Hopper", …} ← body

HTTP/1.1 200 OK                                   ← status line: read this first
Content-Type: application/json; charset=utf-8

{"firstname": "Grace", "lastname": "Hopper", …}
```

2xx — it worked | **3xx** — look elsewhere | **4xx** — your request is wrong

5xx — their bug or outage

⚠ THE QUIRK YOU'LL MEET FIRST

Restful-Booker answers `/ping` with **201** and DELETE with **201**, and uses **403** for a missing token. Test what the contract says, and flag deviations for discussion.

rq requests in one screen

YOU WRITE	IT DOES
`requests.Session()`	keeps connections, cookies and default headers between calls — the `http` fixture
`json=payload`	serialises the dict **and** sets `Content-Type: application/json`
`data=payload`	form-encoded for a dict, raw for a string — the wrong choice for a JSON API
`params={"lastname": "Smith"}`	builds and encodes the query string
`headers=` · `cookies=` · `auth=("user", "pw")`	extra headers · a cookie (Restful-Booker's token) · HTTP Basic
`timeout=10` or `(3.05, 10)`	connect/read limits. **Without it, requests waits forever.**
`r.status_code` · `r.headers` · `r.json()` · `r.text`	status · case-insensitive headers · parsed body · raw body
`r.elapsed` · `r.raise_for_status()`	time to the response headers · raise for 4xx/5xx

The layers an API framework grows into

P04-02 calls requests straight from tests. By P04-04 the same tests sit on three layers, so a URL or auth change is a one-line fix.



WHERE ASSERTIONS LIVE

Business assertions belong in tests. A client may raise on an unexpected transport status to keep positive tests short, but it must also expose the raw response for negative tests.

TOOLKIT — TECHNIQUES

🧰 Nine moves you'll reuse

Most API tests are two or three of these combined.

🚦

Status first, body in the message

The failure explains itself.

When: every call you assert on.

```
r = http.get(f"
{booker_url}/booking/{bid}",
timeout=10)
assert r.status_code == 200, r.text
# body in the message
```

POWERS → **P04-02 · P04-03**

⏱️

A timeout on every call

A hang becomes a fast, clear failure.

When: always — ruff's S113 enforces it.

```
http.get(url, timeout=(3.05, 10))
# (connect, read)
```

POWERS → **every task**

Round trip

Write, read back, compare.

When: proving a create or update really persisted.

```
created = http.post(url, json=payload,
timeout=10).json()
assert http.get(f"
{url}/{created['bookingid']}",
timeout=10).json() == payload
```

POWERS → **P04-02 · P04-03**

Never 5xx

Bad input must be rejected, not crash the server.

When: every negative case.

```
assert r.status_code < 500, r.text
# never 5xx…
assert r.status_code == expected
# …then the exact code
```

POWERS → **P04-03 · P04-08**

Validate into a model

Wrong types fail with the field's name.

When: you consume a response's fields.

```
booking =
Booking.model_validate(r.json())   #
fails loudly on the wrong shape
```

POWERS → **P04-04 · P04-08**

A client per service

Tests read like business statements.

When: the same endpoint appears in more than one test.

```
created =
booker.create_booking(payload)
assert
booker.get_booking(created.bookingid)
== created.booking
```

POWERS → **P04-04 · P05-06 hybrid**

∞

Invariants, not snapshots

Properties that hold whatever the data is.

When: shared demo data can change under you.

```
assert len(ids) == len(set(ids)),
"duplicates across pages"
assert len(ids) == total
```

POWERS → **P04-07**

🔑

Tokens per role

Fetch once per session, one per role.

When: auth and RBAC tests.

```
@pytest.fixture(scope="session")
def tokens(http, settings): ...   #
{"admin": …, "customer": …,
"anonymous": None}
```

POWERS → **P04-03** · **P04-07**

🧵

Correlation IDs

Jump from a failing test to the server's logs.

When: triaging failures across services.

```
headers = {"X-Request-ID": f"
{request.node.name}-
{uuid.uuid4().hex[:8]}"}
```

POWERS → **P04-08**

P04-01 · 🟢 ZERO · 2H

HTTP fundamentals and manual API exploration

App — Restful-Booker

Deliverable — `docs/api-exploration.md`

📋 The task

1. Using **curl** and **Bruno or Postman**, call every Restful-Booker endpoint: `/ping`, `/auth`, `GET /booking`, `GET /booking/{id}`, `POST /booking`, `PUT`, `PATCH`, `DELETE`, and the filters `?firstname=` / `?checkin=`.
2. For each call, record the method, URL, request headers and body, **status code**, response headers (`Content-Type`), a body sample and anything surprising.
3. Write **15 test ideas** in a table, at least 7 of them negative (missing auth, bad dates, wrong types, non-existent IDs…).
4. Save your Bruno/Postman collection in `api-collections/`.

`done when` Tick these off

- [ ] Every endpoint has been called successfully at least once
- [ ] You've noted which status codes differ from REST conventions (there are some; that's a discussion point)

📘 User guide

`concepts` What you need to know

- **Request:** method + URL (scheme, host, **path params** like `/booking/5`, **query params** like `?lastname=Smith`) + headers + optional body.
- **Response:** status code + headers + body.
- **Headers you'll use constantly:** `Content-Type` (what I'm sending), `Accept` (what I want back), `Authorization`, `Cookie`.
- **REST:** resources are nouns (`/booking`), methods are verbs, and the server is **stateless** (each request carries all context, e.g. the token).

`deep dive` Going deeper

Method semantics (interviewers probe these):

METHOD	PURPOSE	SAFE?	IDEMPOTENT?	TYPICAL SUCCESS
GET	Read	✅	✅	200

METHOD	PURPOSE	SAFE?	IDEMPOTENT?	TYPICAL SUCCESS
POST	Create / action	❌	❌	201 (or 200)
PUT	Replace entire resource	❌	✅	200 / 204
PATCH	Partial update	❌	Not guaranteed	200 / 204
DELETE	Remove	❌	✅	204 / 200

Safe = no server state change. *Idempotent* = repeating it has the same effect as doing it once. Idempotency decides what's safe to **retry** (P04-04).

Status code families: `2xx` success, `3xx` redirect, `4xx` **client** error (your request is wrong), `5xx` **server** error (their bug or outage).

CODE	MEANING	TEST ANGLE
400	Malformed request	Broken JSON, wrong types
401	Not authenticated ("who are you?")	Missing or invalid token
403	Authenticated but not allowed ("I know you, but no")	Customer calling an admin endpoint
404	Not found	Deleted or non-existent ID
409	Conflict	Duplicate email, version conflict
422	Semantically invalid	Checkout before checkin
429	Rate limited	Too many requests
500	Server crashed	**Always a bug when caused by client input**

Explore before you automate. Manual exploration builds the mental model your automation encodes. Use heuristics: **CRUD** round trips, **boundaries** (empty, long, unicode, negative), **auth** (none, wrong, expired, wrong role), **formats** (missing fields, extra fields, wrong types), **state** (delete then read, update a deleted resource).

INTERACTIVE — SEND THE SAME REQUEST TWICE

server state before booking 1 = {firstname: "Ada", totalprice: 100}

1st GET → 200 {Ada, 100} state unchanged

2nd GET → 200 {Ada, 100} state unchanged

Same answer, same state: **safe and idempotent** — always fine to retry.

INTERACTIVE — WHICH STATUS CODE SHOULD COME BACK?

1 / 8 · The body isn't valid JSON: `{"firstname": "Ada",`

Pick a status code. Score: 0 / 0

🧪 Worked examples

LEVEL 1 — CRACK IT

🟢 Low: read-only calls with curl.

```
curl -i https://restful-booker.herokuapp.com/ping              # -i shows status + headers
curl -s "https://restful-booker.herokuapp.com/booking?firstname=Sally" | head -c 300
curl -s -H "Accept: application/json" https://restful-booker.herokuapp.com/booking/1
```

SAY THIS

`-i` shows the status line and headers. I read those before the body — the status code is the first assertion.

LEVEL 2 — IMPRESS THEM

🔵 Medium: create, auth, update, with timing.

```
BASE=https://restful-booker.herokuapp.com
TOKEN=$(curl -s -X POST $BASE/auth -H 'Content-Type: application/json' \
  -d '{"username":"admin","password":"password123"}' | python3 -c 'import
sys,json;print(json.load(sys.stdin)["token"])')

ID=$(curl -s -X POST $BASE/booking -H 'Content-Type: application/json' -H 'Accept:
application/json' \
  -d '{"firstname":"Ada","lastname":"Lovelace","totalprice":111,"depositpaid":true,
      "bookingdates":{"checkin":"2026-01-01","checkout":"2026-01-
03"},"additionalneeds":"Tea"}' \
  | python3 -c 'import sys,json;print(json.load(sys.stdin)["bookingid"])')

curl -s -o /dev/null -w "PATCH status=%{http_code} time=%{time_total}s\n" -X PATCH
$BASE/booking/$ID \
  -H 'Content-Type: application/json' -H "Cookie: token=$TOKEN" -d '{"firstname":"Grace"}'
```

HIRE SIGNAL

The token comes from `/auth` and goes back as a cookie; `-w` prints the status and timing without the body.

LEVEL 3 — HERO MODE

🟣 High/Complex: a test-idea matrix derived from heuristics.

#	ENDPOINT	IDEA	TYPE	EXPECTED
1	POST /booking	Valid payload round-trips exactly	Positive	200 + echo
2	POST /booking	`checkout` before `checkin`	Validation	4xx (if 200 → bug?)
3	POST /booking	`totalprice` as string `"abc"`	Type	4xx, never 500
4	POST /booking	Missing `firstname`	Required field	4xx
5	PUT /booking/{id}	No token	Auth	403
6	PUT /booking/{id}	Expired or garbage token	Auth	403
7	PATCH /booking/{id}	Only firstname → other fields unchanged	Partial update	200

#	ENDPOINT	IDEA	TYPE	EXPECTED
8	DELETE /booking/{id}	Then GET the same ID	State	404
9	DELETE /booking/{id}	Twice	Idempotency	2nd → 404/405, not 500
10	GET /booking?checkin=	Invalid date format	Validation	4xx or empty list, not 500
11	POST /booking	10,000-char `additionalneeds`	Boundary	Defined limit or accepted
12	POST /booking	Unicode names `Zoë 李`	i18n	Round-trips unchanged

Mark each idea with what you actually observed. Where "expected" and "actual" differ, you have a **bug candidate** (P10-01).

WHY IT'S HERO-LEVEL

Every idea states its expected result *before* the call. Where expected and actual differ, you have a bug candidate.

🎤 Interview Q&A

Answer out loud in under two minutes **before** you open each one.

Z **PUT vs PATCH?**

PUT replaces the whole resource with the payload (omitted fields may be cleared) and is idempotent. PATCH applies a partial update to specific fields. Test PATCH by asserting that the untouched fields are unchanged.

Z **401 vs 403?**

401 Unauthorized really means *unauthenticated*: no or invalid credentials. 403 Forbidden means authenticated but lacking permission. Some APIs (including Restful-Booker) use 403 for a missing token, so test what the contract says and flag deviations for discussion.

H **What does "idempotent" mean for HTTP methods, and why do testers care?**

Repeating the request has the same effect on server state as sending it once. GET, PUT and DELETE should be idempotent; POST generally isn't. Testers care because (1) it defines what's safe to retry automatically, and (2) it's testable: send PUT twice and the state is the same; send POST twice and you probably get a duplicate (sometimes a bug, e.g. double payments, if there's no idempotency key).

H **400 vs 422?**

400 means the request is malformed (unparseable JSON, wrong content type). 422 means it's well-formed but semantically invalid (checkout before checkin, invalid email). Many APIs use 400 for both. What matters is consistency and never returning 500 for client mistakes.

H **You get an API with no documentation. How do you test it?**

Discover: capture real traffic (browser DevTools, proxy), look for OpenAPI/Swagger endpoints, read the client code. Explore with heuristics (CRUD, auth, boundaries, errors). Document the inferred contract, confirm it with developers, then automate the confirmed behaviour and push for a spec, which becomes the basis for contract tests.

SH **REST vs GraphQL vs gRPC: how does your testing approach change?**

REST: resource URLs, status codes carry meaning, test per endpoint. GraphQL: a single endpoint, errors often come back as HTTP 200 with an `errors` array, so assert the body, test query depth/complexity limits, field-level auth and N+1 performance. gRPC: a binary protobuf contract, status codes are gRPC codes, you test with generated stubs, and contract changes are caught via proto compatibility rules. The principles stay the same: contract, auth, validation, errors, performance.

🤖 AI ASSIST

Paste the API docs page and ask AI for *more negative test ideas*. Add only the ones you can justify.

P04-02 · 🟢 ZERO · 3H

First API tests with requests

App — Restful-Booker

Deliverable — `tests/api/booker/test_booking_basic.py`

📋 The task

Automate the core checks with `requests` (using the `http`, `settings` and `booker_url` fixtures from P03):

1. `/ping` health check (assert the *documented* code).
2. `GET /booking` returns a list of objects with `bookingid`.
3. `POST /booking` creates a booking and echoes the payload, then `GET /booking/{id}` returns the same data (round trip).
4. The filter `?firstname=` returns your created booking.
5. Assert the `Content-Type` header, and put a soft response-time budget (e.g. < 3 s) in the assertion message.
6. **Every call has a timeout.**

`done when` Tick these off

- [ ] ≥6 tests, all green, none depending on pre-existing data
- [ ] Assertion messages include the response body on failure

📘 User guide

`concepts` What you need to know

- `requests.get/post/put/patch/delete(url, params=, json=, data=, headers=, cookies=, auth=, timeout=)`.
- **`json=`** serialises a dict *and* sets `Content-Type: application/json`. **`data=`** sends form-encoded (or raw) data.
- **Response:** `.status_code`, `.ok`, `.json()`, `.text`, `.headers` (case-insensitive dict), `.elapsed`, `.raise_for_status()`.
- **Timeout:** `timeout=10` or `(connect, read)` like `(3.05, 10)`.

`deep dive` Going deeper

No timeout means a hung suite. `requests` waits **forever** by default. One stalled server freezes CI until the job-level timeout (often 6 hours). Ruff rule `S113` from P00-04 catches this. Always pass `timeout`.

What to assert (in order):

1. **Status code.** Assert it first with a message that includes the body: `assert r.status_code == 200, r.text`.
2. **Headers** that matter (`Content-Type`, caching, `Location` for 201s).
3. **Body:** key fields and **types**, not the whole blob.
4. **Side effects:** read back what you wrote (round trip), and later check the DB (P06).

Dynamic fields (IDs, timestamps) make whole-body equality brittle. Compare the fields you control, or strip dynamic keys before comparing.

`response.json()` raises if the body isn't JSON, e.g. an HTML error page from a proxy. Assert the status and content type first so failures are readable.

Response time checks in functional tests should be **coarse smoke budgets** that catch "suddenly 30 s", not performance testing. Real performance testing needs controlled load and percentiles (a separate discipline).

🧪 Worked examples

LEVEL 1 — CRACK IT

🟢 Low: health and list.

tests/api/booker/test_booking_basic.py

```python
import pytest

@pytest.mark.smoke
def test_ping(http, booker_url):
    r = http.get(f"{booker_url}/ping", timeout=10)
    assert r.status_code == 201, r.text          # yes, 201: it's documented, and a classic gotcha

def test_booking_ids_are_listed(http, booker_url):
    r = http.get(f"{booker_url}/booking", timeout=10)
    assert r.status_code == 200
    body = r.json()
    assert isinstance(body, list) and body, "expected a non-empty list"
    assert all("bookingid" in item for item in body)
```

Runs against the simulated Restful-Booker in your browser (see the phase map). Run it against the real API in your repo.

> SAY THIS
>
> *Status first, with the body in the assertion message, then the shape of the body.*

LEVEL 2 — IMPRESS THEM

🔵 Medium: a create → read round trip.

tests/api/booker/test_round_trip.py

```python
from python_basics.checkout import build_booking

def test_created_booking_can_be_read_back(http, booker_url):
    payload = build_booking(firstname="Ada", lastname="Lovelace")
    created = http.post(f"{booker_url}/booking", json=payload,
                        headers={"Accept": "application/json"}, timeout=(3.05, 10))
    assert created.status_code == 200, created.text
    assert created.headers["Content-Type"].startswith("application/json")
    booking_id = created.json()["bookingid"]
    assert created.json()["booking"] == payload

    fetched = http.get(f"{booker_url}/booking/{booking_id}", headers={"Accept":
"application/json"}, timeout=10)
    assert fetched.status_code == 200, fetched.text
    assert fetched.json() == payload
    assert fetched.elapsed.total_seconds() < 3, f"slow response: {fetched.elapsed}"
```

Runs against the simulated Restful-Booker in your browser (see the phase map). Run it against the real API in your repo.

> HIRE SIGNAL
>
> *Create, read back, compare to what I sent: the round trip proves the write really happened, and the timing check is only a coarse budget.*

LEVEL 3 — HERO MODE

🟣 High/Complex: edge-value round trips with helpful diffs.

tests/api/booker/test_edge_values.py

```python
import pytest
from python_basics.checkout import build_booking
from python_basics.drills import json_diff        # P02-06

EDGE_CASES = [
    pytest.param({"firstname": "Zoë", "lastname": "李"}, id="unicode"),
    pytest.param({"totalprice": 0}, id="zero-price"),
    pytest.param({"depositpaid": False}, id="no-deposit"),
    pytest.param({"bookingdates": {"checkin": "2026-02-28", "checkout": "2026-03-01"}},
id="month-boundary"),
    pytest.param({"additionalneeds": "x" * 500}, id="long-text"),
]

@pytest.mark.parametrize("overrides", EDGE_CASES)
def test_booking_round_trips_edge_values(http, booker_url, overrides):
    payload = build_booking(**overrides)
    r = http.post(f"{booker_url}/booking", json=payload, headers={"Accept":
"application/json"}, timeout=10)
    assert r.status_code == 200, f"create failed: {r.status_code} {r.text[:300]}"

    got = http.get(f"{booker_url}/booking/{r.json()['bookingid']}",
                   headers={"Accept": "application/json"}, timeout=10).json()
    diffs = json_diff(payload, got)
    assert not diffs, "round-trip mismatch:\n" + "\n".join(diffs)
```

Runs against the simulated Restful-Booker in your browser (see the phase map). Run it against the real API in your repo.

If an edge case fails, you may have found a real bug. Write it up in P10-01 rather than "fixing" the test.

> WHY IT'S HERO-LEVEL
>
> Edge values are parametrized and compared with a path-reporting diff, so a mismatch names the exact field.

🎤 Interview Q&A

Answer out loud in under two minutes **before** you open each one.

Z `json=` vs `data=` in requests?

`json=` serialises a Python object to JSON and sets `Content-Type: application/json`. `data=` sends form-encoded data for dicts, or raw bytes/strings as-is, and you set the content type yourself. Using `data=dict` against a JSON API is a common bug (the server gets form data and returns 400/415).

Z **Why must every request have a timeout?**

requests has no default timeout, so a hung server blocks the test (and the CI job) indefinitely. A timeout turns a hang into a fast, clear failure. Use a connect/read tuple and a sensible default from config.

H **What do you assert in an API test?**

Status code (with the body in the message), important headers, body fields and types (ideally via schema/model validation), business rules (totals, states), side effects (read back, DB checks), and for errors the error contract (code, message format). I avoid asserting volatile fields (timestamps, IDs) except for format.

H **How do you compare API responses that contain dynamic values?**

Compare only controlled fields, strip or mask dynamic keys before comparing, assert formats for dynamic values (UUID regex, ISO timestamp within a time window), or validate with a schema/model and check the specific business fields separately. A path-reporting diff makes failures readable.

H **Should functional API tests check response time?**

Only as a coarse guardrail (e.g. < 2–3 s per call) to catch severe regressions, with generous limits to avoid flakiness. Real performance validation (percentiles under load, throughput) belongs in dedicated performance tests with controlled environments.

🤖 AI ASSIST

Ask AI to review your test file for *missing assertions* ("what bug would still pass these tests?").

P04-03 · 🔵 HERO · 4H

Auth, CRUD and negative testing

App — Restful-Booker

Deliverable — `tests/api/booker/test_booking_crud.py`, `tests/api/booker/test_booking_negative.py`

📋 The task

1. Full lifecycle test: create → read → PUT (full replace) → PATCH (partial) → DELETE → GET returns 404.
2. After a PATCH, assert that the **untouched fields are unchanged**.
3. Authenticate two ways: the token **cookie** and **HTTP Basic** (`auth=("admin", "password123")`).
4. A parametrized negative matrix: no token, garbage token, non-existent ID, wrong types, missing required fields, malformed JSON.
5. For every negative case, assert **"never 5xx"** as well as the expected code. If the API returns 500 for bad input, mark it `xfail(strict=True)` with a bug ID and write the bug report (P10-01).

`done when` Tick these off

- [ ] Lifecycle and negative tests pass (or strict-xfail with a filed bug)
- [ ] You can explain each negative category and why it matters

📘 User guide

`concepts` What you need to know

- **Authentication** (who you are) vs **authorization** (what you may do).
- **Auth mechanisms:** Basic (base64 `user:pass` in the `Authorization` header), **Bearer token** (`Authorization: Bearer <jwt>`), **API key** (a header or query param), **cookie/session**, **OAuth2** (tokens issued by an auth server).
- **Negative testing:** proving the system *rejects* bad input safely and clearly.

`deep dive` Going deeper

Token lifecycle in a suite: get a token once per session (a session fixture). If tokens expire mid-run (long suites), build a client that refreshes on 401 **once** and then fails. Never hard-code tokens.

Negative test categories:

CATEGORY	EXAMPLES	EXPECTED
Authentication	Missing, garbage, expired token	401/403
Authorization	Customer → admin endpoint, user A → user B's data (**IDOR**)	403/404
Validation	Missing required, wrong type, out-of-range, bad format	400/422 with a clear message
Existence	Unknown ID, already deleted	404
Conflict	Duplicate unique field	409
Protocol	Wrong method, wrong content type, malformed JSON	405/415/400

5xx on bad input is always a bug. It means input wasn't validated, and it often hides security issues. A robust negative test asserts both `status < 500` and the specific expected code, so the failure message is precise.

IDOR (Insecure Direct Object Reference): can user A read or modify user B's resource by changing an ID? It's one of the most common real-world API vulnerabilities, and testers find it with simple two-user tests.

🧪 Worked examples

LEVEL 1 — CRACK IT

🟢 **Low: token plus an authorised update.**

tests/api/booker/test_booking_put.py

```
from python_basics.checkout import build_booking

def test_put_with_token_replaces_booking(http, booker_url, booker_token, booking):
    new = build_booking(firstname="Grace", lastname="Hopper", totalprice=999)
    r = http.put(f"{booker_url}/booking/{booking['bookingid']}", json=new,
                 cookies={"token": booker_token}, headers={"Accept": "application/json"},
timeout=10)
    assert r.status_code == 200, r.text
    assert r.json() == new
```

Runs against the simulated Restful-Booker in your browser (see the phase map). Run it against the real API in your repo.

> SAY THIS
>
> *The token rides in as a cookie. PUT replaces the whole booking, so I compare the whole response.*

LEVEL 2 — IMPRESS THEM

🔵 Medium: PATCH semantics and the full lifecycle.

tests/api/booker/test_booking_crud.py

```
def test_patch_changes_only_given_fields(http, booker_url, booker_token, booking):
    booking_id, before = booking["bookingid"], booking["booking"]
    r = http.patch(f"{booker_url}/booking/{booking_id}", json={"firstname": "Patched"},
                   cookies={"token": booker_token}, headers={"Accept": "application/json"},
timeout=10)
    assert r.status_code == 200, r.text
    after = r.json()
    assert after["firstname"] == "Patched"
    assert {k: v for k, v in after.items() if k != "firstname"} == \
           {k: v for k, v in before.items() if k != "firstname"}

def test_deleted_booking_is_gone(http, booker_url, make_booking):
    booking_id = make_booking()["bookingid"]
    d = http.delete(f"{booker_url}/booking/{booking_id}", auth=("admin", "password123"),
timeout=10)
    assert d.status_code in (200, 201, 204), d.text      # check the docs: Restful-Booker's
code is unusual
    assert http.get(f"{booker_url}/booking/{booking_id}", timeout=10).status_code == 404
```

Runs against the simulated Restful-Booker in your browser (see the phase map). Run it against the real API in your repo.

> HIRE SIGNAL
>
> *After PATCH, every other field must equal the before-state — partial means partial. Delete, then prove it's gone with a 404.*

LEVEL 3 — HERO MODE

🟣 High/Complex: a data-driven negative matrix with a "never 5xx" guard.

tests/api/booker/test_booking_negative.py

```
import pytest
from python_basics.checkout import build_booking

def without(field):
    p = build_booking(); p.pop(field); return p

NEGATIVE = [
    # id,                method,   path,               auth,       body,
expected
    ("no-token",         "PUT",    "/booking/{id}",    None,       build_booking(),
403),
    ("garbage-token",    "PUT",    "/booking/{id}",    "garbage",  build_booking(),
403),
    ("unknown-id",       "GET",    "/booking/999999999", None,     None,
404),
    ("missing-lastname", "POST",   "/booking",         None,       without("lastname"),
400),
    ("price-as-text",    "POST",   "/booking",         None,
build_booking(totalprice="abc"),       400),
    ("malformed-json",   "POST",   "/booking",         None,       '{"firstname": "Ada",',
400),
]

@pytest.mark.parametrize("method,path,auth,body,expected", [pytest.param(*row[1:],
id=row[0]) for row in NEGATIVE])
def test_bad_requests_are_rejected_cleanly(http, booker_url, booking, method, path, auth,
body, expected):
    url = booker_url + path.format(id=booking["bookingid"])
    kwargs = {"timeout": 10, "headers": {"Accept": "application/json", "Content-Type":
"application/json"}}
    if auth:
        kwargs["cookies"] = {"token": auth}
    if isinstance(body, str):
        kwargs["data"] = body                          # raw malformed JSON
    elif body is not None:
        kwargs["json"] = body
    r = http.request(method, url, **kwargs)
    assert r.status_code < 500, f"server error on bad input ({r.status_code}):
{r.text[:200]}"
    assert r.status_code == expected, f"expected {expected}, got {r.status_code}:
{r.text[:200]}"
```

Runs against the simulated Restful-Booker in your browser (see the phase map). Run it against the real API in your repo. Pick `--buggy-booker` to make the simulation answer 500 to bad input, and watch the guard fire.

> NOTE
>
> Run it and **observe**: some of these may return 500 on the demo app. Don't change `expected` to 500. Mark it `xfail(strict=True, reason="BUG-…")` and report it.

WHY IT'S HERO-LEVEL

Each case asserts “never 5xx” first, then the exact code, so a crash and a wrong code fail with different messages.

🎤 Interview Q&A

Answer out loud in under two minutes **before** you open each one.

Z **Authentication vs authorization?**

Authentication verifies identity (login, token validity). Authorization decides what an authenticated identity may access (roles, ownership). Tests must cover both: invalid identity is rejected, and a valid identity can't exceed its permissions.

H **How do you handle token expiry in a long-running suite?**

Fetch the token in a session fixture. Wrap the client so a 401 triggers one re-authentication and retry of that request, and log it. Decode the JWT `exp` claim to refresh proactively. Keep a separate test that verifies expired tokens *are* rejected, so the refresh logic never hides an auth bug.

H **What categories of negative tests do you write for an API?**

Authentication, authorization (roles and IDOR), input validation (required, types, formats, ranges, lengths), existence (unknown or deleted IDs), conflicts (duplicates, stale versions), protocol (method, content type, malformed body), and rate limiting. Each asserts the specific code, a helpful error body, and never 5xx.

H **The API returns 500 when you send a string for a number. Is that a bug?**

Yes. 5xx means the server failed to handle client input, which should be a 4xx with a validation message. It can indicate missing validation and potential security issues, and it pollutes error monitoring. I'd report it with the request, response and expected behaviour, and keep a strict-xfail test linked to the bug so we notice when it's fixed.

SH **How do you test an RBAC permission matrix without writing hundreds of tests by hand?**

Define the matrix as data: roles × endpoints × methods → expected allow/deny. Generate parametrized tests from it (`pytest_generate_tests`), with a token fixture per role and per-test data ownership (user A's resources vs user B's for IDOR). Review the matrix with product/security, since it becomes living documentation, and run it on every permission-related change.

SH **How would you test an OAuth2-protected API in automation?**

Use the client-credentials grant (or a test-only confidential client) to get tokens programmatically. Avoid automating the provider's login UI. Cache tokens per role for the session, and test token handling separately: expired, wrong audience or scope, revoked. Keep client secrets in the CI secret store, and use short-lived tokens in pipelines.

🤖 AI ASSIST

Ask AI: *"List OWASP API Top 10 risks that a functional tester can check with simple requests."* Pick two (e.g. BOLA/IDOR) and add tests.

P04-04 · 🔵 HERO · 4H

API client layer and schema validation

App — Restful-Booker

Deliverable — `framework/api/base_client.py` , `framework/api/booker_client.py` , `framework/api/models/booking.py`

📋 The task

1. `BaseClient`: owns a `requests.Session`, joins base URL + path, applies the default timeout, logs every request and response, and **retries only idempotent methods** on 502/503/504.
2. `BookerClient(BaseClient)`: `auth()`, `create_booking()`, `get_booking()`, `update_booking()`, `patch_booking()`, `delete_booking()`, `list_bookings(**filters)`.
3. Pydantic models `Booking`, `BookingDates`, `BookingCreated`. Successful calls return **models**, and negative tests can still get the raw `Response`.
4. Refactor your P04-02/P04-03 tests to use the client. **No raw URLs in tests.**
5. On test failure, attach the last request/response to the report (use the P03-06 hook pattern).

`done when` Tick these off

- [ ] Tests read like: `booking = booker.create_booking(payload); assert booker.get_booking(booking.bookingid) == booking.booking`
- [ ] A schema change (e.g. `totalprice` returned as a string) makes tests fail with a clear pydantic error

📘 User guide

`concepts` What you need to know

- **Layered framework:** tests (intent and assertions) → clients (endpoints) → transport (session, auth, retries, logging).
- **Service object / API client:** one class per service or resource, one method per operation.
- **Pydantic:** `BaseModel`, `model_validate(dict)`, `model_dump()`, `ConfigDict(extra="forbid")`, validators.
- **Retry adapter:** `HTTPAdapter(max_retries=Retry(...))` from `urllib3`.

`deep dive` Going deeper

Why a client layer? When the base URL, auth scheme or an endpoint path changes, you fix one method, not 200 tests. Tests become readable business statements. Logging and retries are applied consistently.

Where do assertions live? Tests assert **business outcomes**. Clients may **validate transport-level expectations** (e.g. `expect=200` → raise with the body if not) so positive tests stay terse,

but they never decide pass/fail for business rules. For negative tests, expose the raw response (`create_booking_raw()` or `expect=None`).

Schema strictness is a design decision:

MODE	PYDANTIC	CATCHES	RISK
Lenient	`extra="ignore"`	Missing or wrong-typed fields you use	Misses new or unknown fields
Strict	`extra="forbid"`	Any unexpected field (contract drift)	Breaks on harmless additive changes

Common practice: **lenient in functional tests** (validate what you consume) and **strict in dedicated contract tests** (P04-08).

Retries in a test framework: retry *transport* failures (502/503/504, connection resets) on *idempotent* methods only, with backoff. Never retry POST blindly (duplicates), and never retry assertion failures. Log each retry so flakiness is visible.

🧪 Worked examples

LEVEL 1 — CRACK IT

🟢 Low: a minimal base client.

framework/api/base_client.py

```
# framework/api/base_client.py
import requests

class BaseClient:
    def __init__(self, base_url: str, timeout: float = 10.0) -> None:
        self.base_url = base_url.rstrip("/")
        self.timeout = timeout
        self.session = requests.Session()
        self.session.headers.update({"Accept": "application/json"})

    def request(self, method: str, path: str, **kwargs) -> requests.Response:
        kwargs.setdefault("timeout", self.timeout)
        return self.session.request(method, f"{self.base_url}/{path.lstrip('/')}",
**kwargs)
```

SAY THIS

One place owns the base URL, the headers and the timeout; tests pass paths, never full URLs.

LEVEL 2 — IMPRESS THEM

🔵 Medium: a typed client with pydantic models.

framework/api/models/booking.py

```python
# framework/api/models/booking.py
from pydantic import BaseModel, ConfigDict

class BookingDates(BaseModel):
    checkin: str
    checkout: str

class Booking(BaseModel):
    model_config = ConfigDict(extra="ignore")      # lenient: functional tests
    firstname: str
    lastname: str
    totalprice: int
    depositpaid: bool
    bookingdates: BookingDates
    additionalneeds: str | None = None

class BookingCreated(BaseModel):
    bookingid: int
    booking: Booking
```

framework/api/booker_client.py

```python
# framework/api/booker_client.py
import requests
from framework.api.base_client import BaseClient
from framework.api.models.booking import Booking, BookingCreated

class ApiError(AssertionError):
    pass

class BookerClient(BaseClient):
    def _expect(self, r: requests.Response, status: int) -> requests.Response:
        if r.status_code != status:
            raise ApiError(f"{r.request.method} {r.url} → {r.status_code} (expected
{status}): {r.text[:500]}")
        return r

    def auth(self, username: str = "admin", password: str = "password123") -> None:
        r = self._expect(self.request("POST", "/auth", json={"username": username,
"password": password}), 200)
        self.session.cookies.set("token", r.json()["token"])

    def create_booking_raw(self, payload: dict) -> requests.Response:
        return self.request("POST", "/booking", json=payload)

    def create_booking(self, payload: dict) -> BookingCreated:
        return BookingCreated.model_validate(self._expect(self.create_booking_raw(payload),
200).json())

    def get_booking(self, booking_id: int) -> Booking:
        return Booking.model_validate(self._expect(self.request("GET",
f"/booking/{booking_id}"), 200).json())
```

```python
# test using the client
def test_round_trip(booker):              # fixture: BookerClient(settings.booker_url) with
.auth()
    created = booker.create_booking(build_booking(firstname="Ada"))
    assert booker.get_booking(created.bookingid) == created.booking
```

HIRE SIGNAL

Successful calls return validated models, and `create_booking_raw()` keeps the raw response for negative tests.

try it What the models catch — and a trap + ADDED

The second "done when" box says a `totalprice` returned as a string should fail with a clear pydantic error. Run this first: in pydantic's default (lax) mode a **numeric** string like `"150"` is silently converted. You need `strict=True` to catch it.

```
# + added: what the P04-04 models catch - and one trap
from pydantic import ValidationError

good = {"firstname": "Ada", "lastname": "Lovelace", "totalprice": 150, "depositpaid": True,
        "bookingdates": {"checkin": "2026-01-01", "checkout": "2026-01-05"},
"brand_new_field": "?"}
print(Booking.model_validate(good))                        # extra="ignore": the unknown
field is dropped

try:
    Booking.model_validate({**good, "totalprice": "abc"})
except ValidationError as e:
    print("rejected:", e.errors()[0]["loc"], "→", e.errors()[0]["msg"])

trap = Booking.model_validate({**good, "totalprice": "150"})
print("totalprice sent as the STRING '150' →", repr(trap.totalprice), "(lax mode coerces
numeric strings)")


class StrictBooking(Booking):
    model_config = ConfigDict(extra="forbid", strict=True)      # contract-test style (P04-
08)


try:
    StrictBooking.model_validate({**good, "totalprice": "150"})
except ValidationError as e:
    for err in e.errors():
        print("strict:", err["loc"], "→", err["msg"])
```

try it The client layer end to end + ADDED

The models and `BookerClient` above, the full `BaseClient` from 🏆 Hero Mode, and its `booker` fixture — plus three tests. The P03-06 report hook (`-p framework.pytest_plugin`) is what lets the fixture attach the exchanges on failure.

tests/api/booker/test_booker_client.py

```
# tests/api/booker/test_booker_client.py   (+ added: the client layer in action)
from python_basics.checkout import build_booking


def test_round_trip(booker):
    created = booker.create_booking(build_booking(firstname="Ada"))
    assert booker.get_booking(created.bookingid) == created.booking


def test_every_exchange_is_recorded(booker):
    booker.create_booking(build_booking())
    assert any(entry.startswith("POST") for entry in booker.history)


def test_failure_report_carries_the_exchanges(booker):
    created = booker.create_booking(build_booking(lastname="Lovelace"))
    assert booker.get_booking(created.bookingid).lastname == "Hopper", "deliberately wrong
- read the API exchanges below"
```

Runs against the simulated Restful-Booker. The third test fails on purpose: scroll the output to the “API exchanges” section the fixture attached.

LEVEL 3 — HERO MODE

🟣 High/Complex: retries for idempotent methods, exchange capture and report attachment.

framework/api/base_client.py

```
# framework/api/base_client.py (full)
import logging
import requests
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

log = logging.getLogger("api")

class BaseClient:
    def __init__(self, base_url: str, timeout: float = 10.0) -> None:
        self.base_url, self.timeout = base_url.rstrip("/"), timeout
        self.session = requests.Session()
        self.session.headers.update({"Accept": "application/json"})
        retry = Retry(total=3, backoff_factor=0.5, status_forcelist=(502, 503, 504),
                      allowed_methods=frozenset({"GET", "PUT", "DELETE", "HEAD",
"OPTIONS"}),
                      raise_on_status=False)
        self.session.mount("https://", HTTPAdapter(max_retries=retry))
        self.session.mount("http://", HTTPAdapter(max_retries=retry))
        self.session.hooks["response"].append(self._record)
        self.history: list[str] = []                        # last N exchanges for failure
reports

    def _record(self, r: requests.Response, *args, **kwargs) -> None:
        req = r.request
        entry = (f"{req.method} {req.url}\n> {(req.body or b'')[:500]!r}\n"
                 f"< {r.status_code} in {r.elapsed.total_seconds():.2f}s\n<
{r.text[:500]}")
        self.history = (self.history + [entry])[-5:]
        log.info("%s %s → %s (%.2fs)", req.method, req.url, r.status_code,
r.elapsed.total_seconds())

    def request(self, method: str, path: str, **kwargs) -> requests.Response:
        kwargs.setdefault("timeout", self.timeout)
        return self.session.request(method, f"{self.base_url}/{path.lstrip('/')}",
**kwargs)
```

tests/api/conftest.py

```
# tests/api/conftest.py: attach the API history to the report when a test fails
import pytest
from framework.api.booker_client import BookerClient

@pytest.fixture
def booker(settings, request):
    client = BookerClient(settings.booker_url, timeout=settings.http_timeout)
    client.auth()
    yield client
    rep = getattr(request.node, "rep_call", None)          # set by the P03-06 plugin
    if rep is not None and rep.failed:
        rep.sections.append(("API exchanges", "\n\n".join(client.history)))
    client.session.close()
```

WHY IT'S HERO-LEVEL

Retries only for idempotent methods on 502/503/504, and the last five exchanges are attached to the report when a test fails.

🎤 Interview Q&A

Answer out loud in under two minutes **before** you open each one.

H **Why build an API client layer instead of calling requests in tests?**

Single responsibility and change isolation: URLs, auth, headers, timeouts, retries and logging live in one place. Tests become readable business scenarios, and endpoint changes are one-line fixes. It also enables typed models, shared evidence capture and reuse by UI tests (API setup for UI flows).

H **Where should assertions live: in clients or in tests?**

Business assertions live in tests. Clients may enforce expected transport status for convenience (raise with context), but they must also expose raw responses for negative tests. Hiding business checks in clients makes tests unclear about what they verify.

H **Pydantic vs JSON Schema for response validation?**

Pydantic: Python-native models, typed objects for tests, readable errors, and it doubles as the data model. JSON Schema: language-agnostic, can be generated from or shared with OpenAPI specs, and good for pure contract checks. Many frameworks use pydantic models in functional tests and spec-based validation in contract tests.

SH **Strict or lenient schema validation?**

It depends on the goal. Functional tests should be lenient (validate what they use) so additive, backward-compatible changes don't break unrelated tests. Contract tests should be strict (forbid unknown fields, validate every documented field) to detect drift early, and they're owned with the API team. Mixing the two causes either brittleness or blind spots.

SH **How do you make API test failures easy to debug?**

Capture request/response exchanges (method, URL, headers minus secrets, body, status, timing) and attach them to the report on failure. Add a correlation ID header per test so backend logs can be searched. Use assertion messages with the body, and link to logs or traces. Mask tokens and PII in anything logged.

🤖 AI ASSIST

Paste a JSON response and ask AI to *draft* pydantic models, then review the types (int vs float, optional fields) against real responses.

P04-05 · 🟢 ZERO · 3H

Docker as a user: images, containers, ports and volumes

App — The Internet (Docker image)

Deliverable — `docs/docker-cheatsheet.md`

📋 The task

WHY HERE?

From P04-07 onwards you'll run Toolshop, Restful-Booker and Pagila on your own machine, so you need to *use* Docker confidently first. Building and maintaining your own images comes later, in P08.

1. `docker pull gprestes/the-internet`, then **find the port it listens on yourself**: `docker image inspect` (look at `Config.ExposedPorts`).
2. Run it detached with a port mapping, open it in your browser, and point a small smoke check at it (🔵 example). In P05-05 you'll point your UI tests at the same container.
3. Practise the lifecycle: `ps`, `logs -f`, `exec -it … sh`, `stop`, `start`, `rm`, `images`, `rmi`, `system df`.
4. **Volumes:** run `postgres` with a named volume, create a table, remove the container, start a new one on the same volume, and show the table survived.
5. Write `docs/docker-cheatsheet.md` in your own words.

`done when` Tick these off

- [ ] Your smoke check passes against the local container (`QE_INTERNET_URL=http://localhost:7080`)
- [ ] You can explain image vs container, `EXPOSE` vs `-p`, and named volume vs bind mount

 User guide

`concepts` What you need to know

- **Image:** a read-only, layered filesystem plus metadata (default command, env, exposed ports). **Container:** a running (or stopped) instance of an image with its own writable layer.
- **Registry and tags:** `name:tag` (e.g. `postgres:18`). `latest` is just a tag, not "newest guaranteed".
- **Port publishing:** `-p HOST:CONTAINER` maps a host port to the container's port. `EXPOSE` in an image is *documentation only*.
- **Volumes:** a **named volume** is managed by Docker and survives containers. A **bind mount** maps a host folder into the container.
- **Networks:** containers on the same user-defined network reach each other **by name**.

deep dive Going deeper

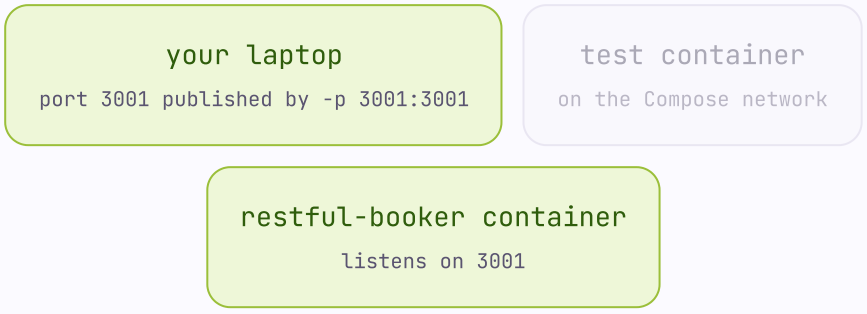
Containers are processes, not VMs. They share the host kernel and are isolated with namespaces and cgroups, so they start in milliseconds and are light. (On macOS and Windows, Docker runs them inside a small Linux VM, which is why file mounts can be slower there.)

Ephemeral by design. Anything written inside a container's writable layer disappears with `docker rm`. Persist data in volumes, and get test reports out through bind mounts (P08-01).

`localhost` means "me". Inside a container, `localhost` is the container itself, not your laptop or another container. This is the number-one Docker networking confusion. Other containers are reached by service or container name on a shared network, and the host by `host.docker.internal` (Docker Desktop).

Why a container "exits immediately": a container lives only as long as its main process. If the command finishes (or crashes), the container stops. `docker logs <name>` tells you why.

INTERACTIVE — WHO IS "LOCALHOST"?



✓ Laptop → `localhost:3001` → the port Docker published with `-p 3001:3001` → the container. **Works.**

🧪 Worked examples

LEVEL 1 — CRACK IT

🟢 Low: pull, inspect, run, observe.

```
docker pull gprestes/the-internet
docker image inspect gprestes/the-internet --format '{{json .Config.ExposedPorts}}'   # find the port
docker run -d --name internet -p 7080:<container-port> gprestes/the-internet
docker ps
docker logs -f internet            # Ctrl+C to stop following
curl -I http://localhost:7080/
```

SAY THIS

Inspect before you run: the image tells you which port it listens on, and `-p HOST:CONTAINER` publishes it.

LEVEL 2 — IMPRESS THEM

🔵 Medium: point a smoke check at the container.

tests/smoke/test_internet_up.py

```python
# tests/smoke/test_internet_up.py (in P05-05 the same fixture moves to tests/ui/internet/conftest.py)
import os

import pytest
import requests


@pytest.fixture(scope="session")
def internet_url() -> str:
    return os.getenv("QE_INTERNET_URL", "https://the-internet.herokuapp.com")


@pytest.mark.parametrize("path", ["/", "/login", "/dynamic_loading", "/nested_frames"])
def test_page_is_served(internet_url: str, path: str) -> None:
    response = requests.get(f"{internet_url}{path}", timeout=10)
    assert response.status_code == 200, f"{path} returned {response.status_code}"
```

```
QE_INTERNET_URL=http://localhost:7080 uv run pytest tests/smoke -q
docker exec -it internet sh -c 'ls /; ps aux | head'   # look inside (tools vary by image)
docker stop internet && docker rm internet
```

HIRE SIGNAL

The test reads its URL from an env var, so the same file checks the public site or my container.

LEVEL 3 — HERO MODE

🟣 High/Complex: persistence, resource limits and inspection with templates.

```
docker volume create pgdata
docker run -d --name pg1 -e POSTGRES_PASSWORD=secret -v pgdata:/var/lib/postgresql -p
5433:5432 postgres:18
docker exec -it pg1 psql -U postgres -c "CREATE TABLE survivor(id int); INSERT INTO
survivor VALUES (42);"
docker rm -f pg1
docker run -d --name pg2 -e POSTGRES_PASSWORD=secret -v pgdata:/var/lib/postgresql -p
5433:5432 postgres:18
docker exec -it pg2 psql -U postgres -c "SELECT * FROM survivor;"     # 42 is still there
# PG 18 images store data under /var/lib/postgresql/<version>; older images used
/var/lib/postgresql/data

docker run -d --name limited --memory 512m --cpus 1 gprestes/the-internet
docker stats --no-stream limited
docker inspect limited --format 'IP={{range .NetworkSettings.Networks}}{{.IPAddress}}
{{end}} Restart={{.HostConfig.RestartPolicy.Name}}'
docker rm -f pg2 limited && docker volume rm pgdata
```

WHY IT'S HERO-LEVEL

Named volumes outlive containers; limits and `inspect --format` show what's really running.

🎤 Interview Q&A

Answer out loud in under two minutes **before** you open each one.

Z **Image vs container?**

An image is an immutable template (layered filesystem plus config). A container is a running instance of it with a thin writable layer. You can run many containers from one image, and deleting a container loses anything not stored in a volume.

Z **Docker container vs virtual machine?**

VMs virtualise hardware and run a full guest OS: heavy, with slow boot and strong isolation. Containers share the host kernel and isolate processes with namespaces and cgroups: light, fast to start, dense. For test environments, containers give reproducible dependencies in seconds.

H **`EXPOSE` vs `-p`?**

`EXPOSE` in a Dockerfile documents which port the app listens on and publishes nothing. `-p 8080:80` (or `ports:` in Compose) actually maps a host port to a container port. Containers on the same network can reach each other's ports without `-p`.

H **Named volumes vs bind mounts?**

Named volumes are managed by Docker, portable, and good for database data that must survive container recreation. Bind mounts map a specific host path, which is good for source code during development and for getting test reports out to the host. Bind mounts depend on host paths and permissions.

H **Inside a container, tests fail to reach `http://localhost:3001`, which works on your laptop. Why?**

Inside the container, localhost is the container itself. Use the other container's service name on a shared network (`http://restful-booker:3001`), or `host.docker.internal` to reach a service on the host (Docker Desktop, or with an extra host mapping on Linux).

🤖 AI ASSIST

Paste a failing `docker run` error and ask AI to explain it, then confirm with `docker logs` and `docker inspect`.

P04-06 · ZERO · 3H

Run the demo stack locally with Compose

App — Toolshop, Restful-Booker, Pagila

Deliverable — `docs/local-stack.md`, `Makefile` targets `stack-up`, `stack-down`, `stack-reset`

The task

1. Clone and start **Toolshop**, **Restful-Booker** and **Pagila** with their own Compose files (DEMO_APPS.md). Seed the Toolshop DB.
2. Use `docker compose ps`, `logs -f <service>`, `exec`, `stop`, `down` and `down -v` (understand what `-v` destroys).
3. Resolve at least one **port conflict** (e.g. something already on 5432 or 8080) by changing the *host* side of a mapping in an override file.
4. Run your suites with `--env local`.
5. Add `make stack-up` / `stack-down` / `stack-reset` plus a readiness check, and document everything in `docs/local-stack.md`.

`done when` Tick these off

- [ ] `make stack-up && uv run pytest --env local -m "smoke"` works from a cold start
- [ ] `docs/local-stack.md` has start, stop, reset and troubleshooting sections

User guide

`concepts` What you need to know

- **Compose file:** declares **services** (containers), networks and volumes for a multi-container app.
- **Project:** the set of containers from one compose file, named after the folder or `name:`. It gets its own default network.
- **Commands:** `up -d`, `ps`, `logs`, `exec`, `stop`, `down` (removes containers and networks), `down -v` (**also deletes volumes**).
- **Override file:** `docker-compose.override.yml` is merged automatically, so you can change ports locally without editing upstream files.

`deep dive` Going deeper

Service discovery: Compose puts a project's services on one network where each service is reachable by its **service name** (`http://laravel-api:80` inside the network, even while the host sees `localhost:8091`). Host ports are only for *you* (and host processes).

State and resets: DB data usually lives in a named volume, so `down` keeps your data and `down -v` wipes it. Know which one you need. A deterministic reset command (Toolshop's `migrate:fresh --seed`) is gold for testing: reset → run → known state.

Readiness vs started: "container started" ≠ "app ready". Databases need seconds to accept connections and web apps need time to boot. Always wait on a *readiness signal* (an HTTP 200 on a health URL, or a DB query) before running tests. P08-02 builds this into Compose with health checks.

🧪 Worked examples

LEVEL 1 — CRACK IT

🟢 Low: everyday Compose commands.

```
cd ~/src/practice-software-testing && docker compose up -d
docker compose ps
docker compose logs -f --tail=50          # all services; add a service name to focus
docker exec -it pst-laravel-api-1 php artisan migrate:fresh --seed     # reset data (check container name)
docker compose down                        # keep volumes
```

SAY THIS

`up -d`, `ps`, `logs -f`, `down` — and `down -v` only when I mean to wipe the data.

LEVEL 2 — IMPRESS THEM

🔵 Medium: Makefile orchestration across stacks.

```
STACKS := ~/src/practice-software-testing ~/src/restful-booker ~/src/pagila

stack-up:
	@for d in $(STACKS); do (cd $$d && docker compose up -d); done
	@uv run python scripts/wait_for_stack.py

stack-down:
	@for d in $(STACKS); do (cd $$d && docker compose down); done

stack-reset:
	docker exec pst-laravel-api-1 php artisan migrate:fresh --seed
```

> HIRE SIGNAL
>
> *One target starts every stack and then waits for readiness, not just for "container started".*

LEVEL 3 — HERO MODE

🟣 High/Complex: a readiness checker that tells you *which* service isn't ready.

scripts/wait_for_stack.py

```
# scripts/wait_for_stack.py
import sys
import requests
from framework.utils.wait import wait_until       # P02-05

CHECKS = {
    "toolshop-api": "http://localhost:8091/products",
    "toolshop-ui": "http://localhost:4200/",
    "restful-booker": "http://localhost:3001/ping",
}

def ready(url: str) -> bool:
    try:
        return requests.get(url, timeout=3).status_code < 500
    except requests.RequestException:
        return False

def main() -> int:
    failed = []
    for name, url in CHECKS.items():
        try:
            wait_until(lambda: ready(url), timeout=180, interval=3, message=f"{name} at
{url}")
            print(f"✅ {name}")
        except TimeoutError as e:
            print(f"❌ {e}")
            failed.append(name)
    return 1 if failed else 0

if __name__ == "__main__":
    sys.exit(main())
```

```
# docker-compose.override.yml in the Pagila folder: fix a port conflict on the HOST side
only
services:
  pagila:                  # check the service name with: docker compose config --services
    ports: !override
      - "5544:5432"
```

NOTE

`!override` replaces the list instead of merging it (a recent Compose feature). If your version lacks it, edit the port in a copy of the file.

WHY IT'S HERO-LEVEL

The checker names the service that isn't ready, and the override changes the port on the host side only.

🎤 Interview Q&A

Answer out loud in under two minutes **before** you open each one.

Z **`docker compose` vs `docker run`?**

`docker run` starts one container with flags on the command line. Compose declares a multi-container application (services, networks, volumes, env, dependencies) in a versioned YAML file, starts it with one command and gives shared networking with service-name DNS. It's reproducible and reviewable.

H **How do services in a Compose project talk to each other?**

Compose creates a network per project, and each service is resolvable by its service name. Service A calls `http://service-b:<container-port>`. Host port mappings are irrelevant between containers and only needed for access from the host.

H **`docker compose down` vs `down -v`?**

`down` removes containers and networks but keeps named volumes (your DB data survives). `down -v` also deletes the volumes, which is a full data reset. That's handy for clean test runs and disastrous if you meant to keep the data.

Z **You get "port is already allocated". What do you do?**

Another process or container already uses that host port. Find it (`docker ps` , `lsof -i :5432`), stop it, or map a different *host* port (`"5544:5432"`) via an override file, then update your test config (`.env`).

H **How do you give every test run a known environment state?**

A scripted reset: recreate volumes (`down -v && up`), run the app's seed/migrate command, or restore a DB snapshot or template, then wait for readiness. Tests still create their own data, and the reset guarantees the *reference* data (users, catalogue) is known.

🤖 AI ASSIST

Ask AI to explain an unfamiliar service in Toolshop's compose file (what it is and why it's there), then verify by reading the file and the logs.

P04-07 · 🔵 HERO · 4H

Toolshop API: bearer auth, pagination, filters, data factories

App — Toolshop API

Deliverable — `framework/api/toolshop_client.py` , `framework/data/factories.py` , `tests/api/toolshop/`

📋 The task

Use Swagger (`/api/documentation`) to discover endpoints, then:

1. **Login** (`POST /users/login`) returns a bearer token. `GET /users/me` returns the right user. Decode the JWT and assert the claims you find (e.g. expiry in the future).

2. **Pagination invariants** on `GET /products`: the items across all pages equal `total`, there are **no duplicates** across pages, and a page beyond the last returns empty or a defined error.
3. **Filtering and search:** every result matches the filter (category, brand, or search term).
4. **Factories:** `make_user()` / `make_brand()` with Faker plus a unique suffix, and cleanup where the API allows it.
5. **RBAC:** for one admin-only operation, assert admin succeeds and customer and anonymous are rejected (discover the codes, and flag inconsistencies).

`done when` Tick these off

- [ ] Tests pass against **hosted** *and* **self-hosted** Toolshop (`--env public` / `--env local`)
- [ ] No test depends on product names or counts that could change; assert invariants instead

 User guide

`concepts` What you need to know

- **Bearer token / JWT:** `header.payload.signature`, base64url-encoded JSON with claims like `sub`, `exp` and `role`. Anyone can *decode* it, but only the server can *verify* it.
- **Pagination styles:** page/offset (`?page=2`) and cursor (`?after=abc`). Metadata: `total`, `per_page`, `last_page`.
- **Invariants:** properties that must always hold regardless of data (all results match the filter, and so on).
- **Factories + Faker:** generate realistic, unique data on demand.

`deep dive` Going deeper

Test invariants, not snapshots. Shared demo data changes. Instead of "page 1 has these 9 products", assert *properties*: every item has required fields, pages don't overlap, the union's size equals `total`, and sorting holds. This is how you write tests that survive data changes.

Pagination bugs to hunt: duplicates or missing items at page boundaries (unstable sort!), `total` disagreeing with the actual count, page `0` /negative/non-numeric pages returning 500, oversized `per_page` not capped.

Filter testing: positive (all results match), negative (non-matching items excluded, checked by creating one), empty results (a valid filter with no matches → empty list, not an error), invalid filter values (→ 4xx, not 500), and combined filters (AND semantics).

Data factories: `Faker` makes realistic data. Add a **unique suffix** (uuid or worker ID, P07) so parallel tests never collide. Seed Faker (`Faker.seed(n)`) when you want reproducible runs. Cleanup options, from best to worst: delete via API → use a disposable environment → mark with a prefix and sweep periodically.

INTERACTIVE — DECODE A JWT (IN YOUR BROWSER, NOTHING IS SENT)

HEADER

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

PAYLOAD (CLAIMS)

```
{
  "sub": "2",
  "role": "user",
  "iat": 1791965117,
  "exp": 1791965417
}
```

✓ `exp` is 5 min in the future — the P04-07 assertion passes

INTERACTIVE — WALK EVERY PAGE AND CHECK THE INVARIANTS

PAGE 1 · FETCHED	PAGE 2 · FETCHED	PAGE 3 · FETCHED
#1 · $5	#4 · $9	#7 · $20
#2 · $9	#5 · $12	
#3 · $9	#6 · $15	

collected ids: **[1, 2, 3, 4, 5, 6, 7]** total reported: **7**

len(ids) == total? **yes** · duplicates: **none** · missing: **none** — every invariant holds.

🧪 Worked examples

LEVEL 1 — CRACK IT

🟢 Low: login and "who am I".

```
def test_customer_can_login_and_see_profile(http, settings):
    api = settings.toolshop_api_url
    r = http.post(f"{api}/users/login", json={"email":
"customer@practicesoftwaretesting.com",
                                              "password": "welcome01"}, timeout=10)
    assert r.status_code == 200, r.text
    token = r.json()["access_token"]
    me = http.get(f"{api}/users/me", headers={"Authorization": f"Bearer {token}"},
timeout=10)
    assert me.status_code == 200, me.text
    assert me.json()["email"] == "customer@practicesoftwaretesting.com"
```

SAY THIS

Login returns a bearer token; `/users/me` proves the token belongs to the user who logged in.

LEVEL 2 — IMPRESS THEM

🔵 Medium: role tokens, pagination invariants and JWT claims.

tests/api/toolshop/conftest.py

```python
# tests/api/toolshop/conftest.py
import pytest

USERS = {"admin": ("admin@practicesoftwaretesting.com", "welcome01"),
         "customer": ("customer@practicesoftwaretesting.com", "welcome01")}

@pytest.fixture(scope="session")
def tokens(http, settings) -> dict[str, str | None]:
    out: dict[str, str | None] = {"anonymous": None}
    for role, (email, password) in USERS.items():
        r = http.post(f"{settings.toolshop_api_url}/users/login",
                      json={"email": email, "password": password}, timeout=10)
        r.raise_for_status()
        out[role] = r.json()["access_token"]
    return out

@pytest.fixture(scope="session")
def admin_token(tokens):
    return tokens["admin"]

@pytest.fixture(scope="session")
def customer_token(tokens):
    return tokens["customer"]
```

tests/api/toolshop/test_products.py

```python
# tests/api/toolshop/test_products.py
import base64, json, time
from framework.api.pagination import iter_products      # P02-07

def jwt_claims(token: str) -> dict:
    payload = token.split(".")[1]
    return json.loads(base64.urlsafe_b64decode(payload + "=" * (-len(payload) % 4)))

def test_token_expires_in_future(customer_token):
    assert jwt_claims(customer_token)["exp"] > time.time()

def test_pagination_has_no_gaps_or_duplicates(http, settings):
    api = settings.toolshop_api_url
    total = http.get(f"{api}/products", timeout=10).json()["total"]
    ids = [p["id"] for p in iter_products(http, api)]
    assert len(ids) == len(set(ids)), "duplicate products across pages"
    assert len(ids) == total, f"walked {len(ids)} items but API reports total={total}"
```

HIRE SIGNAL

Tokens per role are fetched once per session; pagination is checked with invariants — no duplicates, and the union equals `total`.

try it The JWT check, offline + ADDED

```python
# + added: decode a JWT the way the P04-07 test does
import base64, hashlib, hmac, json, time


def b64url(data: bytes) -> str:
    return base64.urlsafe_b64encode(data).rstrip(b"=").decode()


header = {"alg": "HS256", "typ": "JWT"}
claims = {"sub": "2", "role": "user", "iat": int(time.time()), "exp": int(time.time()) +
300}
signing_input = f"{b64url(json.dumps(header).encode())}.
{b64url(json.dumps(claims).encode())}"
token = signing_input + "." + b64url(hmac.new(b"server-secret", signing_input.encode(),
hashlib.sha256).digest())
print("token:", token[:52] + "…")


def jwt_claims(token: str) -> dict:                        # exactly as in the P04-07
example
    payload = token.split(".")[1]
    return json.loads(base64.urlsafe_b64decode(payload + "=" * (-len(payload) % 4)))


print("claims:", jwt_claims(token))
assert jwt_claims(token)["exp"] > time.time()              # the P04-07 assertion

# Anyone can DECODE the claims. Only the server, which holds the secret, can VERIFY the
signature:
parts = token.split(".")
parts[1] = b64url(json.dumps({**claims, "role": "admin"}).encode())
forged = ".".join(parts)
recomputed = b64url(hmac.new(b"server-secret", ".".join(parts[:2]).encode(),
hashlib.sha256).digest())
print("forged role:", jwt_claims(forged)["role"], "| does the old signature still match?",
recomputed == parts[2])
```

try it Why "no duplicates" matters as much as "len == total" + ADDED

```python
# + added: the P04-07 pagination invariants, against a stable and an unstable API
PRODUCTS = [{"id": i, "price": p} for i, p in enumerate([5, 9, 9, 9, 12, 15, 20], start=1)]


class Api:
    """Pages of 3, sorted by price. Three products cost 9: ties are where pagination
breaks."""

    def __init__(self, stable: bool) -> None:
        self.stable, self.calls = stable, 0

    def page(self, n: int, size: int = 3) -> dict:
        self.calls += 1
        if self.stable:
            ordered = sorted(PRODUCTS, key=lambda p: (p["price"], p["id"]))         # tie-
breaker: id
        else:                                          # the database returns ties in a
different order per query
            flip = self.calls % 2 == 0
            ordered = sorted(PRODUCTS, key=lambda p: (p["price"], -p["id"] if flip else
p["id"]))
        start = (n - 1) * size
        return {"data": ordered[start:start + size], "total": len(PRODUCTS), "last_page": -
(-len(PRODUCTS) // size)}


def walk(api: Api) -> list[int]:
    first = api.page(1)
    ids = [p["id"] for p in first["data"]]
    for n in range(2, first["last_page"] + 1):
        ids += [p["id"] for p in api.page(n)["data"]]
    return ids


for stable in (True, False):
    ids = walk(Api(stable))
    dupes = sorted({i for i in ids if ids.count(i) > 1})
    missing = sorted({p["id"] for p in PRODUCTS} - set(ids))
    print(f"{'stable  ' if stable else 'unstable'} ids={ids} duplicates={dupes} missing=
{missing} "
          f"len(ids)==total: {len(ids) == len(PRODUCTS)}")
```

LEVEL 3 — HERO MODE

🟣 High/Complex: factories, admin CRUD with cleanup and an RBAC matrix.

framework/data/factories.py

```python
# framework/data/factories.py
import uuid
from faker import Faker

fake = Faker()

def unique(prefix: str) -> str:
    return f"{prefix}-{uuid.uuid4().hex[:8]}"

def brand_payload(**overrides) -> dict:
    name = unique("qe-brand")
    return {"name": name, "slug": name} | overrides

def user_payload(**overrides) -> dict:
    return {
        "first_name": fake.first_name(), "last_name": fake.last_name(),
        "email": f"{unique('qe')}@example.com", "password": "Sup3r$ecret!2026",
        "dob": "1990-01-01", "phone": fake.msisdn()[:10],
        "address": {"street": fake.street_address(), "city": fake.city(), "state":
fake.state(),
                    "country": "NL", "postal_code": "1234AB"},
    } | overrides
```

NOTE

The payload fields must match Toolshop's current register-user schema, so **check Swagger** and adapt (older versions use flat address fields). Fixing a factory from the spec is part of the task.

tests/api/toolshop/test_brands_rbac.py

```python
# tests/api/toolshop/test_brands_rbac.py
import pytest
from framework.data.factories import brand_payload

@pytest.fixture
def created_brands(http, settings, admin_token):
    ids = []
    yield ids
    for brand_id in ids:
        http.delete(f"{settings.toolshop_api_url}/brands/{brand_id}",
                    headers={"Authorization": f"Bearer {admin_token}"}, timeout=10)

@pytest.mark.parametrize("role, allowed", [("admin", True), ("customer", False),
("anonymous", False)])
def test_create_brand_permissions(http, settings, tokens, created_brands, role, allowed):
    headers = {"Authorization": f"Bearer {tokens[role]}"} if tokens.get(role) else {}
    r = http.post(f"{settings.toolshop_api_url}/brands", json=brand_payload(),
headers=headers, timeout=10)
    if r.status_code in (200, 201):
        created_brands.append(r.json()["id"])
    if allowed:
        assert r.status_code in (200, 201), r.text
    else:
        assert r.status_code in (401, 403), f"{role} should be denied but got
{r.status_code}: {r.text[:200]}"
```

If a role that *should* be denied succeeds, you've found an authorization bug, or learned that the endpoint is public in this version. Either way, document it.

WHY IT'S HERO-LEVEL

Factories make unique data, cleanup is tracked, and the RBAC matrix is data: one parametrized test per role.

🎤 Interview Q&A

Answer out loud in under two minutes **before** you open each one.

H **What is a JWT, and what would you test around it?**

A signed token (`header.payload.signature`) carrying claims such as subject, role and expiry. Tests: a valid token grants access; an expired token is rejected; a tampered payload (edit the claim, keep the signature) is rejected; the `alg: none` trick is rejected; the token doesn't contain sensitive data (it's only encoded, not encrypted); role claims match server-side authorization; logout/revocation behaves as specified.

H **How do you test pagination?**

Invariants: the union of all pages equals `total`, with no duplicates or gaps (watch unstable sort order), page size is honoured and capped, and the first, last and beyond-last pages behave as specified. Invalid page parameters (0, negative, text) return 4xx and never 500. Sort order holds across page boundaries. Also test consistency when data changes between page fetches, if the API promises that.

H **How do you test search and filter endpoints?**

Create known data, then verify that every result matches the criteria, known matches are included, known non-matches are excluded, an empty match returns an empty list, invalid values return 4xx, combinations use the right logic (AND/OR), and case/partial matching follows the spec. Assert invariants rather than exact counts on shared data.

SH **How do you keep API tests independent of existing environment data?**

Each test creates what it needs via factories (unique suffixes for parallel safety) and cleans up. Assertions use invariants rather than hard-coded counts. Reference data (roles, product catalogue) comes from a known seed. For destructive or global tests, use isolated environments (containers, P08). Tag and sweep leftovers periodically.

SH **How do you systematically test authorization (RBAC) across an API?**

Build a roles × operations matrix with expected outcomes (with the product owner or security team), obtain tokens per role in fixtures, generate parametrized tests from the matrix, include ownership checks (IDOR: user B accessing user A's resources), run it on every auth-related change, and treat unexpected "allowed" results as high-severity bugs.

🤖 AI ASSIST

Give AI the Swagger JSON of *one* endpoint and ask it to generate the pydantic model and 8 test ideas. Then check both against real responses.

P04-08 · SUPER HERO · 5H

Contract testing and API resilience

App — Petstore v3 (self-hosted), Toolshop specs

Deliverable — `tests/contract/`, `scripts/spec_diff.py`, `docs/contract-testing.md`

The task

1. **Spec coverage:** load the Petstore OpenAPI spec, list all operations, and report which ones your tests call.
2. **Property-based conformance:** run **Schemathesis** against *self-hosted* Petstore (`docker run -p 8080:8080 swaggerapi/petstore3:unstable`) for GET operations. Record failures and classify each as a spec bug or an implementation bug.
3. **Breaking-change detection:** fetch the OpenAPI JSON of two Toolshop sprint versions (find the spec URL via the Swagger UI's network tab), then write `scripts/spec_diff.py` to report removed paths/methods and removed or retyped response fields. Compare with `oasdiff`.
4. **Resilience:** add an `X-Request-ID` header (test node ID + uuid) to every client request, and write `docs/contract-testing.md` explaining schema-based vs consumer-driven contracts (Pact) and when you'd use each.

`done when` Tick these off

- [ ] A spec-coverage report exists (e.g. "12/19 operations covered")
- [ ] At least one real finding (or a documented "none found") from Schemathesis
- [ ] `spec_diff.py` lists differences between two versions

User guide

concepts What you need to know

- **API contract:** the agreed request/response shape and behaviour, often written as an **OpenAPI** spec.
- **Schema/provider contract testing:** verify that the implementation matches the spec.
- **Consumer-driven contracts (Pact):** each consumer records what it *actually uses*, and the provider verifies against all consumers' expectations.
- **Breaking change:** removing an endpoint or field, changing a type, adding a *required* request field, narrowing an enum. **Non-breaking:** adding optional fields or endpoints.
- **Property-based testing:** generate many inputs from a description (the spec) and check properties (no 5xx, response matches schema).

deep dive Going deeper

Why contract tests? In microservices, end-to-end environments are slow and flaky. Contract tests catch integration breaks **at PR time, per service**, without deploying everything. The spec becomes an executable agreement.

Schema-based vs consumer-driven:

	SCHEMA/PROVIDER (OPENAPI)	CONSUMER-DRIVEN (PACT)
Source of truth	Provider's spec	Consumers' recorded expectations
Catches	Implementation ≠ spec	Provider change breaks a *real* consumer
Blind spot	Spec may not reflect real usage	Needs consumer teams to participate
Good for	Public APIs, many unknown consumers	Internal microservices with known consumers

Schemathesis reads the spec, generates valid *and* edge-case requests (via Hypothesis), and checks responses: no 5xx, status codes documented, bodies conform to schemas, content types match. It finds bugs humans don't think of, such as an unexpected `null`, huge integers, or odd unicode.

Breaking-change gates: run a spec diff (e.g. `oasdiff breaking old.json new.json`) in the provider's CI. Breaking changes require a version bump or explicit approval.

Correlation IDs: send a unique `X-Request-ID` per test so a failing test can be matched to server logs and traces. It's cheap, and it pays off in every triage session.

🧪 Worked examples

LEVEL 1 — CRACK IT

🟢 Low: spec coverage report.

tests/contract/test_spec_coverage.py

```
# tests/contract/test_spec_coverage.py
import requests

SPEC_URL = "http://localhost:8080/api/v3/openapi.json"        # self-hosted Petstore
TESTED = {("GET", "/pet/{petId}"), ("POST", "/pet"), ("GET", "/pet/findByStatus")}   # keep
in sync with tests

def test_report_spec_coverage():
    spec = requests.get(SPEC_URL, timeout=10).json()
    ops = {(m.upper(), path) for path, item in spec["paths"].items()
           for m in item if m in {"get", "post", "put", "patch", "delete"}}
    missing = sorted(ops - TESTED)
    print(f"\nSpec coverage: {len(ops & TESTED)}/{len(ops)} operations")
    print("Not covered:\n  " + "\n  ".join(f"{m} {p}" for m, p in missing))
    assert TESTED ⩽ ops, f"tests call undocumented operations: {TESTED - ops}"
```

> SAY THIS
>
> *Spec coverage is a set difference: the operations in the spec minus the operations my tests call.*

LEVEL 2 — IMPRESS THEM

🔵 Medium: property-based conformance with Schemathesis.

tests/contract/test_petstore_conformance.py

```
# tests/contract/test_petstore_conformance.py      (uv add --dev schemathesis)
import pytest
import schemathesis

schema = schemathesis.openapi.from_url("http://localhost:8080/api/v3/openapi.json")   #
v3.x: schemathesis.from_uri(...)

@schema.parametrize()
def test_petstore_conforms_to_spec(case):
    if case.method ≠ "GET":
        pytest.skip("read-only run: don't mutate shared data")
    case.call_and_validate()          # checks: no 5xx, documented status, schema-
conformant body
```

NOTE

Schemathesis's API evolves, so check its docs for your installed version. Run it only against *self-hosted* services: it sends many requests.

HIRE SIGNAL

Schemathesis generates requests from the spec and checks no 5xx, documented status codes and schema-valid bodies.

LEVEL 3 — HERO MODE

🟣 **High/Complex: a breaking-change detector between API versions.**

scripts/spec_diff.py

```python
# scripts/spec_diff.py: usage: python scripts/spec_diff.py old.json new.json
import json, sys

METHODS = {"get", "post", "put", "patch", "delete"}

def operations(spec: dict) -> dict[tuple[str, str], dict]:
    return {(m.upper(), path): op for path, item in spec.get("paths", {}).items()
            for m, op in item.items() if m in METHODS}

def props(spec: dict, schema: dict) -> dict[str, str]:
    """Top-level response properties → type, following local $refs like
#/components/schemas/Product."""
    if "$ref" in schema:
        name = schema["$ref"].split("/")[-1]
        schema = spec["components"]["schemas"][name]
    if schema.get("type") == "array":
        return props(spec, schema.get("items", {}))
    return {k: v.get("type", "ref") for k, v in schema.get("properties", {}).items()}

def success_schema(op: dict) -> dict:
    for code in ("200", "201"):
        content = op.get("responses", {}).get(code, {}).get("content", {})
        if "application/json" in content:
            return content["application/json"].get("schema", {})
    return {}

def main(old_path: str, new_path: str) -> int:
    old, new = (json.load(open(p)) for p in (old_path, new_path))
    old_ops, new_ops = operations(old), operations(new)
    breaking = [f"REMOVED operation {m} {p}" for (m, p) in sorted(old_ops.keys() -
new_ops.keys())]
    for key in sorted(old_ops.keys() & new_ops.keys()):
        before = props(old, success_schema(old_ops[key]))
        after = props(new, success_schema(new_ops[key]))
        for field in sorted(before.keys() - after.keys()):
            breaking.append(f"REMOVED field '{field}' from {key[0]} {key[1]} response")
        for field in sorted(before.keys() & after.keys()):
            if before[field] != after[field]:
                breaking.append(f"RETYPED '{field}' {before[field]}→{after[field]} in
{key[0]} {key[1]}")
    added = sorted(new_ops.keys() - old_ops.keys())
    print("\n".join(breaking) or "No breaking changes detected")
    print(f"(+{len(added)} new operations)")
    return 1 if breaking else 0

if __name__ == "__main__":
    sys.exit(main(*sys.argv[1:3]))
```

```bash
# Cross-check with a dedicated tool
docker run --rm -v "$PWD:/specs" tufin/oasdiff breaking /specs/old.json /specs/new.json
```

Use the script's exit code as a CI gate on the provider side (P09).

WHY IT'S HERO-LEVEL

The detector compares operations and response fields between versions, and its exit code gates the provider's CI.

try it The detector on two tiny specs + ADDED

```
# + added: run spec_diff.py on two tiny specs
import json

from scripts.spec_diff import main

product = {"type": "object", "properties": {"id": {"type": "integer"}, "name": {"type":
"string"},
                                            "price": {"type": "number"}}}
ok = {"200": {"content": {"application/json": {"schema": {"type": "array",
                                                          "items": {"$ref":
"#/components/schemas/Product"}}}}}}
old = {"paths": {"/products": {"get": {"responses": ok}}, "/brands": {"get": {"responses":
{}}}},
       "components": {"schemas": {"Product": product}}}

new = json.loads(json.dumps(old))
del new["paths"]["/brands"]                                                    # removed
operation
new["components"]["schemas"]["Product"]["properties"]["price"] = {"type": "string"}  #
retyped field
del new["components"]["schemas"]["Product"]["properties"]["name"]                    #
removed field
new["paths"]["/categories"] = {"get": {"responses": {}}}                             #
added: not breaking

for name, spec in (("old.json", old), ("new.json", new)):
    with open(name, "w") as f:
        json.dump(spec, f)
print("exit code:", main("old.json", "new.json"))
```

🎤 Interview Q&A

Answer out loud in under two minutes **before** you open each one.

H **What is contract testing and why use it?**

Verifying that a provider and its consumers agree on the API's shape and behaviour, without running the full system. It catches integration breaks early (at PR time) and cheaply, reduces reliance on slow, flaky end-to-end environments, and documents the agreement as executable checks.

SH **Schema-based vs consumer-driven contract testing?**

Schema-based validates the provider against its OpenAPI spec. It's simple, suits public APIs, but only proves conformance to the spec. Consumer-driven (Pact) has each consumer publish the interactions it relies on, and the provider verifies all of them, so it proves real consumers won't break. It needs cross-team adoption and a broker. Many organisations use both: spec validation plus Pact for critical internal consumers.

SH **Which API changes are breaking?**

Removing or renaming endpoints or fields, changing types or formats, making optional request fields required, adding new required request fields, narrowing enums or validation ranges, changing auth requirements, and changing status-code semantics. Usually non-breaking: adding optional request fields, new response fields (if consumers are tolerant readers), new endpoints. Gate breaking changes with a spec diff in CI and version the API.

SH **What is property-based API testing, and when does it beat example-based tests?**

A tool (e.g. Schemathesis with Hypothesis) generates many requests from the schema, including edge cases, and checks universal properties: no 5xx, responses conform to the schema, documented status codes. It beats hand-written examples at finding unexpected input-handling bugs and spec drift. It complements, rather than replaces, business-rule tests, and should target isolated environments.

H **What is a correlation ID and why add it in tests?**

A unique ID sent with each request (e.g. `X-Request-ID`) that services propagate into logs and traces. Including the test name in it lets you jump from a failing test straight to the exact backend logs, which dramatically speeds up triage, especially across microservices.

SH **How do you test a service whose dependency is unavailable or expensive?**

Contract-test the boundary, and use service virtualisation or mocks (WireMock, Prism from the OpenAPI spec, or `responses` / `requests-mock` at the client level) to simulate normal, slow, error and malformed responses. Verify resilience behaviours (timeouts, retries, circuit breaker, fallbacks), and run a small number of real integration tests in a shared environment.

🤖 AI ASSIST

Give AI two spec versions and ask for a breaking-change summary, then compare it with your script's and `oasdiff`'s output. Note where AI missed or invented changes.

🏁 PHASE CHECKPOINT — P04

Prove it without help

⏱️ Live-coding self-test (40 min, no AI)

from an empty folder, write a Restful-Booker test suite with a session fixture for the token, a factory fixture with cleanup, one CRUD lifecycle test and a 4-case negative matrix.

- [ ] Done from memory, without AI, inside 40 minutes

40-MINUTE TIMER

40:00

`mock interview` **AI mock interviewer prompt**

```
Act as an interviewer for an API test automation role (Python + requests + pytest).
Ask about HTTP semantics, status codes, auth, negative testing, framework design,
schema validation and contract testing. 8 questions, one at a time, then one
design question: "design an API test framework for 20 microservices".
Score each 1-5 and tell me what a senior answer would add.
```

Record your results in `interview-notes/P04.md`.

BEYOND P04 — + ADDED

🧩 Four gaps worth closing

P04 uses pydantic for schemas, mentions mocks and rate limits in passing, and assumes you know what requests puts on the wire. These four make that concrete, and each one runs here.

01 JSON Schema, the language-neutral contract

The P04-04 Q&A compares pydantic with JSON Schema. Here is the JSON Schema side: one schema, **every** violation reported with its JSON path — handy when the schema comes straight from an OpenAPI spec.

```python
# + added: validate a response against a JSON Schema
from jsonschema import Draft202012Validator

BOOKING_SCHEMA = {
    "type": "object",
    "required": ["firstname", "lastname", "totalprice", "depositpaid", "bookingdates"],
    "properties": {
        "firstname": {"type": "string", "minLength": 1},
        "lastname": {"type": "string", "minLength": 1},
        "totalprice": {"type": "integer", "minimum": 0},
        "depositpaid": {"type": "boolean"},
        "bookingdates": {"type": "object", "required": ["checkin", "checkout"],
                         "properties": {"checkin": {"type": "string", "format": "date"},
                                        "checkout": {"type": "string", "format": "date"}}},
    },
    "additionalProperties": False,
}

response = {"firstname": "Ada", "lastname": "", "totalprice": "150", "depositpaid": True,
            "bookingdates": {"checkin": "2026-01-01"}, "vip": True}

errors = sorted(Draft202012Validator(BOOKING_SCHEMA).iter_errors(response), key=lambda e:
list(e.path))
for error in errors:                                    # every problem, each with its JSON
path
    print(f"$.{'.'.join(map(str, error.path)) or '(root)'}: {error.message}")
```

02 Fake the transport, not the client

To unit-test client code (retries, rate limits, error handling) without a server, mount a **custom adapter** on a session — the same trick the simulated Restful-Booker uses, and what libraries like `responses` and `requests-mock` do for you. This one also handles **429 Too Many Requests** by honouring `Retry-After` within a budget.

tests/api/test_retry_after.py

```
# tests/api/test_retry_after.py   (+ added)
import time

import pytest
import requests
from requests.adapters import HTTPAdapter
from requests.structures import CaseInsensitiveDict


class ScriptedAdapter(HTTPAdapter):
    """Answers from a script instead of the network: the trick behind this page's simulated
API."""

    def __init__(self, *responses):
        super().__init__()
        self.script, self.seen = list(responses), []

    def send(self, request, **kwargs):
        self.seen.append(f"{request.method} {request.url}")
        status, headers = self.script.pop(0)
        response = requests.Response()
        response.status_code, response.headers = status, CaseInsensitiveDict(headers)
        response._content, response.url, response.request = b"{}", request.url, request
        return response


def get_honouring_retry_after(session, url, max_wait=5.0, sleep=time.sleep):
    r = session.get(url, timeout=10)
    if r.status_code == 429:
        wait = float(r.headers.get("Retry-After", "1"))
        if wait > max_wait:
            raise AssertionError(f"server asked for {wait}s - over the {max_wait}s budget")
        sleep(wait)
        r = session.get(url, timeout=10)
    return r


def session_with(*responses):
    session, adapter = requests.Session(), ScriptedAdapter(*responses)
    session.mount("https://", adapter)
    return session, adapter


def test_waits_as_told_then_succeeds():
    session, adapter = session_with((429, {"Retry-After": "2"}), (200, {}))
    waits = []
    r = get_honouring_retry_after(session, "https://api.example.com/products",
sleep=waits.append)
    assert r.status_code == 200 and waits == [2.0] and len(adapter.seen) == 2


def test_refuses_to_wait_too_long():
    session, _ = session_with((429, {"Retry-After": "120"}))
    with pytest.raises(AssertionError, match="over the 5.0s budget"):
        get_honouring_retry_after(session, "https://api.example.com/products", sleep=lambda
s: None)
```

H **How should a test client handle 429?**

Respect `Retry-After` (seconds or an HTTP date), cap the total wait, and log it so throttling is visible. Repeated 429s in a suite usually mean the tests are too aggressive for a shared environment: lower parallelism against hosted apps, or use a local stack.

03 See exactly what requests will send

`requests.Request(...).prepare()` builds the final request without sending it: query encoding, `json=` vs `data=`, Basic auth, cookies and multipart uploads, all visible.

```
# + added: see exactly what requests will send - no network needed
import requests


def show(req: requests.Request) -> None:
    p = req.prepare()
    print(p.method, p.url)
    for key in ("Content-Type", "Authorization", "Cookie"):
        if key in p.headers:
            print(f"   {key}: {p.headers[key][:70]}")
    body = p.body if p.body is None or isinstance(p.body, str) else p.body[:90]
    print("   body:", body, "\n")


url = "https://restful-booker.herokuapp.com/booking"
show(requests.Request("GET", url, params={"firstname": "Zoë Ann", "tag": ["a", "b"]}))   #
encoding, repeated keys
show(requests.Request("POST", url, json={"firstname": "Ada"}))                           #
json= sets the header
show(requests.Request("POST", url, data={"firstname": "Ada"}))                           #
data= is form-encoded!
show(requests.Request("PUT", url + "/1", json={}, auth=("admin", "password123")))        #
Basic auth
show(requests.Request("PUT", url + "/1", json={}, cookies={"token": "4f1c2a9b"}))        #
token cookie
show(requests.Request("POST", "https://example.com/upload",
                      files={"file": ("users.csv", b"username\nada\n", "text/csv")}))   #
multipart upload
```

04 From curl to requests

DevTools and Bruno can both copy a request as curl. Translating it is mechanical:

CURL	REQUESTS
`-X PATCH`	`http.patch(...)` or `http.request("PATCH", ...)`
`-H 'Content-Type: application/json' -d '{…}'`	`json={…}` (sets the header for you)
`-H "Cookie: token=$TOKEN"`	`cookies={"token": token}`
`-u admin:password123`	`auth=("admin", "password123")`
`"…/booking?firstname=Sally"`	`params={"firstname": "Sally"}`
`-i` · `-w "%{http_code} %{time_total}"`	`r.headers` · `r.status_code`, `r.elapsed`
`--max-time 10`	`timeout=10`

FIELD GUIDE

🎓 Pattern recognition

The situation → what to reach for.

`a test hangs CI for hours`

timeout on every call

ruff S113 catches the missing one

`"it returned 500 for bad input"`

a bug, every time

strict xfail + bug report, never expect 500

`is it safe to retry?`

idempotent methods only

GET, PUT, DELETE on 502/503/504

`did PATCH really only patch?`

compare the other fields

to the before-state

`shared data keeps changing`

invariants over snapshots

no dupes, union = total, all match

`user A can read user B's order`

IDOR / BOLA

two-user tests, high severity

`works on my laptop, not in the container`

localhost is the container

use the service name

`a field was renamed and a consumer broke`

spec diff in provider CI

breaking changes need a version

`"which server log is my test?"`

X-Request-ID per test

test name + uuid

THE GOLDEN RULE

Assert the status first, with the body in the message — then the data. And bad input must never earn a 5xx.

Built from *learning-path/phases/P04-api-requests.md* · QE learning path, zero → hero

🐍 next: P05 — UI testing with Playwright, using these clients for setup